

SynqedAir — Engineering Build Brief

For: Dev 1 (Flutter) and Dev 2 (Node.js backend) —
start date Sept 1

Read this first: the framework decision

You can't build one app in two frameworks. Flutter and React Native produce separate, incompatible codebases — running both in parallel means building the product twice. Here's how the two of you split cleanly instead:

- **Dev 1 builds the actual mobile app in Flutter.**
Flutter is a complete cross-platform toolkit (one codebase, iOS + Android), so this is a full solution on its own — nothing is missing by not using React Native.
- **Dev 2's React Native background is really a JavaScript/TypeScript**

background, and that maps directly onto the **Node.js backend** –

the AI advisor orchestration, flight aggregator integration, and payments.

This was already Dev 2's role in the original plan; nothing changes except which mobile framework Dev 1 is writing the app in.

Nobody has to learn a new stack. The two of you talk to each other through one HTTP API – see the contract below – so you can build in parallel without blocking each other.

What's in this handoff

- `synqedair_flutter/` – Flutter starter project for Dev 1. Installs clean, three real screens (Home, Results, Companion) built to the approved design, wired for an API base URL, currently running on mock data.
- `synqedair-backend/` – Node.js + TypeScript starter for Dev 2. Installs clean, type-checks clean, boots and responds – verified before this was sent to you, not just written and assumed to work.

Real Claude, Duffel,
and Paystack clients are implemented; auth is
stubbed (see its README).

- This document — architecture, API contract, and the week-by-week plan.

Architecture

```
Flutter app (Dev 1) <--- HTTPS/JSON --->
Node.js backend (Dev 2)

|

|--- Claude API (AI advisor parsing)

|--- Duffel API (flight search)

|--- Paystack API (payments)

|--- Postgres (bookings, users)
```

The Flutter app never talks to Claude, Duffel, or Paystack directly — it only ever calls the Node backend. This keeps API keys off the device and means the ranking logic, fee calculation, and pricing rules live in one place, not duplicated between two clients.

API contract (v0.1)

Both starter projects already implement this — it's documented here so it

stays a shared reference as it evolves. Full types:

`src/types/index.ts`

(backend) and `lib/models/*.dart` (Flutter).

Endpoint	Method	Purpose
<code>/api/advisor/parse</code>	POST	Natural language → structured search params
<code>/api/flights/search</code>	GET	Ranked flight offers with full fee breakdown
<code>/api/bookings</code>	POST	Create a booking, get a Paystack payment link

Endpoint	Method	Purpose
<code>/api/auth/request-otp</code>	POST	Send a phone OTP (stubbed)
<code>/api/auth/verify-otp</code>	POST	Verify OTP, get a session token (stubbed)

Rule for changing this contract: whoever changes a field updates both `src/types/index.ts` and the matching Dart model in the same PR. A field rename on one side and not the other is the #1 way two people building in parallel end up blocked on each other in week 6.

Local setup

Dev 2 (backend) – do this first, Dev 1 depends on it running:

```
cd syngedair-backend
npm install
cp .env.example .env          # fill in real
API keys (see README for where to get them)
```

```
npm run dev #  
http://localhost:4000
```

Dev 1 (Flutter):

```
cd synqedair_flutter  
flutter pub get  
flutter run --dart-  
define=API_BASE_URL=http://localhost:4000
```

If Dev 1 is on a physical device rather than a simulator, `localhost` won't reach a backend running on Dev 2's machine — point `API_BASE_URL` at whatever's actually running (a shared dev server, or `ngrok` / similar tunnel) once you're past day-one local testing.

Week-by-week (maps to the Sept–Dec plan from the investor deck)

September — Foundation

- Wk1: [Founder] vendor sign-ups (Duffel, Paystack KYC, Claude API, Apple/Google dev accounts — KYC alone can take 1–3 weeks, start immediately). [Dev2] backend repo running with real keys.

- Wk2–4: [Dev1] wire the three screens to real API calls instead of mock data. [Dev2] auth (real OTP via Termii or Africa's Talking), Postgres schema for users/bookings.

October — Core integrations

- Wk5–6: [Dev2] verify Duffel coverage for your launch corridors' carriers, confirm exact request/response shape against current docs. [Dev1] loading/error states for search.
- Wk6–7: [Dev2] tune the AI advisor prompt against real queries. [Dev1] handle low-confidence responses (`clarifyingQuestion`) in the advisor UI.
- Wk7–8: [Dev2] Paystack webhook — verify payment before creating the Duffel order, never before. [Dev1] payment webview/redirect flow.

November — Trust layer & hardening

- Wk9: [Dev2] e-ticket delivery, PNR retrieval. [Dev1] booking confirmation screen.
- Wk10: [Both] Companion v1 — check-in reminders, hardcoded visa/eTA rules for launch corridors, push notifications.
- Wk11: [Dev2] WhatsApp Business API handoff for human support. [Dev1] support entry point in Companion.

- Wk12: [Both] security pass — confirm no card data touches the backend (Paystack tokenized checkout handles this if wired correctly).

December — Beta, submission, launch

- Wk13: Closed beta, real users, real money.
- Wk14: Bug fixes, store metadata.
- Wk15 (by Dec 12): Submit to Apple/Google — ahead of Apple's Dec 20–27 review slowdown.
- Wk17: Public launch Dec 31 — app already live and stable before that date.

One thing to verify before either of you writes production logic against it

The Duffel client in the backend starter is written from documented API patterns but **hasn't been run against a real Duffel account** — neither has the Paystack client been run against a real transaction. Both are structured correctly and type-check clean, but confirm exact field names against current docs (duffel.com/docs, paystack.com/docs) in week 1 before building more on top of them. APIs shift; don't

assume the field names here
are still exact by the time you're reading this.